SCENARIO 1: Random reboots at cold temperature under stress

In the overnight stress farm (CPU + GPU + memory stress mix, thermal chamber cycling 0°C to 45°C), 6 of 120 units rebooted or panicked.

- **Failures occur roughly once every 4 to 10 hours, mostly when the chamber is at or below 10°C.**
- **Signatures vary from run to run: sometimes "Unable to handle kernel paging request", sometimes an undefined-instruction Oops, and sometimes a watchdog reset with no kernel log.**
- **The same build runs clean for 7 days on the other 114 units.**
- **A scheduler patch also landed in this build.**
  How to debug this scenario for an Android device?

Debugging cold-temperature random reboots on a unit subset

What the symptoms point to

Three facts point away from a pure software bug and toward hardware marginality that the new build may have exposed.

1. **The crash signatures vary.** Paging faults, undefined instructions, and silent watchdog resets don't share a code path. When one root cause produces random corruption of instruction fetch, data, or pointers, the usual culprits are CPU voltage or timing margin, DRAM bit errors, or rail droop.
2. **Only 6 of 120 units fail, and the other 114 are clean for 7 days.** A software race would normally show up across the whole fleet, even if rarely. Failures clustered on a few units suggest unit-specific silicon, DDR, or board variation.
3. **Failures happen at or below 10°C.** Modern FinFET SoCs show temperature inversion: transistors get slower at cold, so the minimum stable voltage (Vmin) rises. A unit near its Vmin limit at room temperature can fall over at cold. DDR timing and PMIC transient response also degrade at cold.

The scheduler patch is most likely a trigger, not the root cause. For example, it could change task placement so that more big cores ramp up together or wake in sync, which raises current spikes and voltage droop on the CPU rail. Treat it as a variable to isolate, not the prime suspect.

Phase 1: Make every crash leave evidence

The watchdog resets with no log are the biggest gap, so fix observability first.

- **Enable crash capture.** Turn on pstore/ramoops (console and pmsg), plus a full RAM dump on panic and on watchdog. On Qualcomm that means download mode; on MediaTek, mrdump/AEE; on Exynos, debug-snapshot.

- **Set the watchdog to bark before it bites.** The bark interrupt can then dump every CPU's registers before the hard reset.
- **Read the reset-reason registers on every boot.** These are the PMIC power-on/power-off reason registers (PON/POFF), the PS_HOLD status, and watchdog bark/bite flags. Also log any PMIC undervoltage or overcurrent faults. A PMIC fault on the CPU or DDR rail would immediately explain the silent resets.
- **Capture what the CPUs were doing at the hang.** If the platform supports it, enable CoreSight ETM/ETB tracing or the per-core last-PC registers (DBGPCSR) so a hard hang still tells you each core's last instruction.
- **Log operating context continuously** to a ring buffer that survives in the RAM dump:
    - cpufreq and devfreq (frequency) transitions per cluster
    - CPU, GPU, and DDR frequencies
    - thermal-zone temperatures
    - which CPU each crash happened on

Phase 2: Mine the crashes you already have

For each Oops, decode the PC and the faulting address against vmlinux, then look for patterns.

- **Paging-request faults.** Compare the bad pointer with what it should have been. A difference of one or two bits strongly suggests a hardware bit-flip rather than a use-after-free.
- **Undefined-instruction faults.** Read the instruction word at that PC from the RAM dump and compare it with vmlinux.
    - If the word in memory is correct but the CPU executed garbage, suspect the CPU core itself: fetch, I-cache, or voltage/timing.
    - If the word in memory is actually corrupted, suspect DRAM errors or a stray DMA write.
- **Look for clustering.** Check whether crashes cluster on one core, one cluster (especially big or prime cores), a particular top frequency step (OPP), or moments of heavy GPU and CPU load together. Clustering on one core or the top OPP is a strong sign of a voltage-margin problem.

Phase 3: Separate hardware from software with A/B swaps

Experiment	If failures stop	If failures continue
Previous build (no scheduler patch) on the 6 failing units, at cold	The patch exposes the problem, e.g. through load pattern or current spikes	It's an existing unit marginality
New build with only the scheduler patch reverted	Same conclusion, narrowed to the patch	Look elsewhere in the build diff
Re-run the 6 failing units repeatedly	—	Repeat offenders confirm the cause is unit-specific
Hold the chamber at 0°C instead of cycling	—	Faster reproduction, giving a shorter feedback loop

Also diff the device tree, voltage tables, and PMIC/CPR configuration between builds. A changed cold-temperature voltage boost, AVS/CPR setting, or regulator mode can be hidden inside a "scheduler" release.

Phase 4: Margin experiments on the failing units

CPU and GPU voltage

- Raise VDD_CPU (and VDD_GPU, if it's shared or adjacent) by about 25–50 mV. If failures disappear, you've confirmed a voltage-margin problem.
- Cap the maximum frequency, or remove the top OPPs, for each cluster in turn.
- Take cores offline one at a time to find a single weak core.
- Check that the cold-temperature voltage adjustment in CPR/AVS actually triggers at the temperature the sensor reports, and that the sensor reads correctly at cold.

Power delivery

- Put an oscilloscope on VDD_CPU, VDD_GPU, VDD_DDR/VDDQ, and VDD_MX/CX, and trigger on undershoot during load steps.
- Compare droop with the scheduler patch on and off. Synchronized wakeups across cores could produce bigger load steps.
- Force the buck converters into PWM mode instead of auto PFM/PWM switching to see if mode transitions are the cause.
- Note that capacitor ESR rises at cold, which can make droop worse.

DDR

- Run stressapptest or memtester at cold, and also run the bootloader's DDR stress test.
- Compare per-unit DDR training results and eye margins between the failing 6 and the passing units.
- Try forcing retraining, lowering the DDR frequency, or adjusting refresh and timing parameters.

Localize the component

Keep the chamber warm and spot-cool one part at a time with a thermal stream or freeze spray: the SoC, then the DDR package (if it isn't stacked on the SoC), then the PMIC. Whichever part reproduces the failure when cooled is the one to focus on.

Phase 5: Correlate the population

Pull manufacturing data for all 120 units:

- SoC speed bin and leakage fuses, CPR/PVS values
- DDR vendor and lot
- PMIC revision
- Board revision and assembly line
- Factory test results

If the 6 failing units share a slow or low-leakage bin, a DDR lot, or a board revision, you have both the root cause and a way to screen for it.

Phase 6: Don't drop the software path entirely

Run a review and a debug build of the scheduler patch in parallel. Use lockdep, KASAN, and, where possible, KCSAN, plus extra debug list checks. Keep in mind that a debug build changes timing and power draw, so a clean result there doesn't fully clear the patch.

Likely outcome and fix

The most probable finding is that the 6 units are low-margin parts. Cold, combined CPU+GPU stress, and possibly the changed load pattern from the scheduler push them below their voltage or timing margin.

Fixes would include:

- Correcting the cold-temperature voltage boost or the voltage tables
- Adjusting PMIC compensation and regulator mode
- Changing DDR timing
- Adding a cold Vmin screen in factory test
- If needed, tuning the scheduler patch to avoid synchronized wakeups across cores

Before closing the issue, confirm the fix on the failing units at 0°C for at least 5–10 times their previous mean time between failures.

Does Regulator and voltage dump helps in this scenario

Yes, a regulator and voltage dump helps, but mainly for finding **wrong settings**, not for catching the event that actually crashes the unit. It's worth knowing what it can and can't show before you rely on it.

What a dump can tell you

Whether the cold voltage adjustment is applied. At ≤10°C the SoC should be running at a higher voltage than at room temperature for the same frequency (the cold boost in CPR/AVS). Compare the actual voltage per frequency step on failing versus passing units, and at cold versus warm. If the failing units sit at the same voltage cold as warm, or lower than passing units at the same frequency, that is a strong lead.

Per-unit differences from adaptive voltage. CPR/AVS gives each chip its own voltage based on its silicon. A slow or low-margin chip may be given too little headroom. Dumping the CPR/AVS state (target voltage, margin, closed-loop corrections, the temperature band in use) from all 120 units and comparing the 6 failing ones against the rest is one of the most useful things you can do.

Whether something changed between builds. Diff the regulator configuration of the old build against the new one. Check minimum and maximum voltage limits, regulator mode (auto PFM/PWM vs forced PWM), ramp rate, and which regulators are kept on. A quiet change here can hide in a release labelled "scheduler patch".

The settings at the moment of the crash. If the voltage state is saved into the RAM dump or pstore, you can see the commanded voltage, frequency, and temperature for each CPU cluster just before the panic. That ties each crash to a specific operating point.

Latched PMIC fault flags. These are the most valuable part. The PMIC records undervoltage, overcurrent, short-circuit, thermal, and power-on/power-off reasons, and these flags survive the reset. A latched undervoltage or overcurrent on the CPU, GPU, or DDR rail would directly explain the watchdog resets with no kernel log. Read them in the bootloader on every boot and log them.

Where to get the data

- `/sys/kernel/debug/regulator/regulator_summary`, plus the per-regulator debugfs entries: who is using each rail, the voltage it was asked for, and its limits.
- The frequency/voltage tables (OPPs) and the cpufreq and devfreq state for each CPU cluster, the GPU, and DDR.
- The vendor's adaptive-voltage debug interfaces:
    - **Qualcomm:** CPR3/CPR4/CPRh and RPMh/AOP state
    - **MediaTek:** PMIC and CPU DVFS (CPUEB) dumps
    - **Exynos:** ECT/ASV tables
- Raw PMIC register dumps, especially the fault, status, and power-on/power-off reason registers.
- The PMIC's own voltage readout (its ADC), where available, to compare the measured voltage with the commanded one.

What it cannot tell you

A software dump is a slow snapshot, taken milliseconds apart at best. The dangerous voltage drops during sudden load steps last nanoseconds to microseconds. If the CPU rail briefly dips below Vmin when several big cores and the GPU wake together, the regulator registers will still show the correct target voltage, and the dump will look clean.

To catch those transients you need:

- **An oscilloscope** on VDD_CPU, VDD_GPU, and the DDR rails, triggered on undershoot, comparing behaviour with and without the scheduler patch.
- **On-chip droop detectors or voltage monitors**, if the SoC has them, with their counters logged and saved into the crash dump.
- **A margin test**: add about +25–50 mV to the CPU or GPU rail. If failures stop, voltage margin is confirmed, whatever the dump showed.

Practical recommendation

Add three things to every boot and every crash dump: the CPR/AVS and regulator state, the frequency and temperature context, and the latched PMIC fault and reset-reason flags. Collect these across all 120 units, cold and warm, and compare the 6 failing units against the 114 passing ones.

Treat a clean dump as inconclusive. It rules out wrong static settings, but not transient droop. For that, rely on the scope captures and the voltage-margin experiments.

SCENARIO 2: Hang or SError during suspend/resume stress

You are assigned an issue on a Pixel DVT build. During suspend/resume stress testing (rapid cycling with scheduled wakeups), about 1 in 300 cycles ends in a device hang or a panic with "SError Interrupt" or "synchronous external abort" in the resume path. Some failures show only a watchdog bite with no log.

- **It began appearing in last week's kernel drop.**
- **It happens on all units at a similar rate, regardless of temperature or lot.**
- **The first-level triage note says: "bus timeout, likely NoC/interconnect issue, assign to the silicon interconnect team."**
- **Crash stacks show register accesses inside the driver for one multimedia IP block.**

How to debug this scenario

Debugging a suspend/resume SError on a Pixel DVT build

Start by challenging the triage note

The evidence points to a kernel software regression, not a NoC or silicon defect:

- **It started with last week's kernel drop.** The silicon didn't change, the software did.
- **It's uniform across units, lots and temperatures.** Marginal silicon or interconnect timing usually varies with process corner and temperature. A fixed rate across the fleet looks like a timing race in code.
- **The stacks always land in one multimedia driver.** A real interconnect bug would show up across many initiators and targets.
- **SError, synchronous external abort and silent hangs fit one known pattern.** The CPU is accessing an IP block's registers while that block is powered off, clock-gated, held in reset, or isolated behind a disabled interconnect port. If the bus returns an error response (DECERR/SLVERR), you get an abort or SError. If the access never completes, the CPU stalls. Then nothing can log, and the watchdog bites.

"Bus timeout" is the symptom, not the cause. Keep ownership with the kernel/driver side, and ask the interconnect team for tooling help, not a handoff.

Step 1: Bisect the kernel drop

Diff the two drops in every area that touches this block's power lifecycle:

- the driver itself
- its device tree node (power-domains, clocks, resets, interconnects, iommus)
- genpd and the power-domain driver
- the clock driver
- the interconnect/bandwidth-vote framework
- the IOMMU/SysMMU driver

- the runtime PM core
- any change enabling async suspend (`device_enable_async_suspend`)
- any EL3/TZ firmware interface change

Then automate a bisect with the stress harness. At a 1/300 failure rate, a "good" verdict needs enough cycles to mean something. The chance of 900 clean cycles when the bug is present is (299/300)^900 ≈ 5%. So run roughly 1,000–1,500 cycles per bisect step before calling a commit good.

Step 2: Extract maximum information from each crash

For synchronous external aborts, decode ESR_EL1. If FAR is valid, map the faulting address against the memory map. Confirm it falls in this IP's register region, and note which register.

SError is asynchronous and imprecise. The reported PC can be well past the offending write. For debugging, convert `writel_relaxed` to `writel` and add a `dsb(sy)` after writes in the suspend/resume paths. This pulls the abort closer to the real access.

Read the interconnect's error logger registers (dump them in the panic handler or from a ramdump). They typically record the initiator, the target and the response type. If the target is this IP and the logger flags it as powered down or unclocked, that is your proof. This is exactly where the interconnect team's expertise helps.

For watchdog-only failures, make sure the watchdog bite triggers a full ramdump through the bootloader. Then get:

- the per-CPU last PC from the ramdump or the CPU debug registers
- pstore/ramoops with ftrace in persistent RAM
- `last_kmsg`

A CPU stuck on a load or store into this IP's address range confirms the same root cause as the SError cases.

Step 3: Instrument the suspend/resume timeline

Enable:

- `no_console_suspend`, `initcall_debug`, `pm_debug_messages`, `pm_print_times`
- trace events: `power:device_pm_callback_start/end`, `rpm_*`, genpd power on/off, `clk_enable/clk_disable`, and interconnect set events
- a trace marker in every register access helper in this driver

Capture the timeline into persistent RAM. Compare a failing cycle against passing ones. You're looking for a register access that lands after the domain turned off or the clock was disabled, or before they're restored on resume.

The most effective single change is a guard in the driver's accessor. It turns a silent hang into a clean stack trace:

```
static inline u32 mmip_read(struct mmip *ip, u32 off)
{
```

```
    WARN_ONCE(pm_runtime_status_suspended(ip->dev) ||
              !__clk_is_enabled(ip->core_clk),
              "mmip: reg access 0x%x while powered down\n", off);
    return readl(ip->base + off);
}
```

With this in place, the next failure tells you exactly which code path made the bad access.

Step 4: Test the likely root causes

With scheduled wakeups and a race at about 1 in 300, these are the usual suspects, roughly in order of likelihood:

1. **An interrupt handler runs while the block is off.** The IRQ isn't disabled across suspend, or a wake IRQ fires during the noirq phase or early resume, before the domain or clocks are restored. Check `disable_irq`/`enable_irq_wake` usage, `IRQF_NO_SUSPEND`, and whether the handler checks power state.
2. **Deferred work isn't stopped before suspend.** A workqueue item, delayed work, hrtimer or kthread survives suspend and touches registers after the domain is cut. Check that every one is cancelled or flushed in the suspend callback.
3. **Supplier ordering broke.** A missing device link to a supplier (the clock controller, the IOMMU, a parent power domain), combined with newly enabled async suspend, lets the supplier suspend before this consumer or resume after it.
4. **The driver's PM callbacks are wired incorrectly.** Problems with `pm_runtime_force_suspend`/`force_resume` usage, a phase mismatch (resume in `.resume` while genpd powers on in noirq, or the reverse), or a runtime PM usage-count imbalance.
5. **Secure firmware hasn't restored access.** EL3/TZ must reprogram an access-control or firewall setting after resume. If the kernel touches the block first, it gets an error response.
6. **The IP's own DMA or embedded microcontroller is still active** when its IOMMU or domain goes down.

Quick experiments to split these hypotheses:

- **`echo 0 > /sys/power/pm_async`.** If the failures vanish, suspect ordering (cause 3).
- **Keep the power domain always-on** (pin the runtime PM usage count, or mark the genpd always-on). If the failures vanish, this confirms an access to an unpowered block.
- **Unbind or blacklist the driver.** If the failures vanish, this confirms the driver is responsible.

Raise the reproduction rate so each experiment finishes faster: shorten the wake interval, add jittered delays in the driver's suspend and resume callbacks, and keep the IP busy (an active encode or decode) when suspend starts.

Step 5: Fix and validate

Apply the fix that matches the cause: correct the callback phase, add the device link, disable the IRQ properly, cancel the pending work, or fix the firmware restore order. Keep the accessor guard as a debug-config assertion.

For validation, run at least 3,000 clean cycles. At the old 1/300 rate, the chance of passing that many by luck is (299/300)^3000 ≈ e^{-10}, effectively zero. For DVT sign-off, 5,000–10,000 cycles across several units is better.

When the silicon team should own it

Escalate to the interconnect team only if traces prove all of the following during the faulting access, and it still errors:

- the domain was on
- the clocks were enabled
- the reset was deasserted
- the interconnect vote was in place
- the firewall was configured

If that's the case, hand them the error-logger dumps, the timeline and the bisect result, so they start from evidence rather than from "bus timeout".

How devicetree is parsed in Android

On Android, the devicetree passes through three stages: it's built at compile time, then the bootloader selects and merges it, then the kernel parses it. Android userspace also reads a small part of it.

1. Build time: DTS → DTB/DTBO

Hardware is described in `.dts`/`.dtsi` source files. The devicetree compiler (`dtc`) turns them into a binary **Flattened Device Tree (FDT)**, the `.dtb` file. Android splits this into two parts as part of the Treble model:

- **Base DTB** is SoC-level and maintained by the silicon vendor. On older devices it is appended to the kernel or stored in the boot image. With boot image header v2 it goes in the `dtb` section of `boot.img`, and from v3/v4 onward it lives in `vendor_boot`.
- **DTBO** holds the board- and ODM-specific overlays. It is compiled with `dtc -@` so the symbols can be resolved later, then packed with `mkdtboimg.py` into the `dtbo` partition. That image has a `dt_table_header` (magic `0xd7b7ab1e`) followed by `dt_table_entry` records. Each record carries an `id`, a `rev` and custom fields that the bootloader uses for matching.

2. Bootloader: select, overlay, fix up

The bootloader (Qualcomm ABL/LK, U-Boot, etc.) does the first real "parsing". It uses libfdt and Android's **libufdt**.

1. It loads the base DTB and picks the right one when several exist. On Qualcomm this means matching `qcom,msm-id` and `qcom,board-id` against the hardware.

2. It reads the `dtbo` partition header and selects the matching overlay entries by id/rev.
3. It merges the overlays into the base tree with `ufdt_apply_overlay()`. Fragments target nodes through `__symbols__`, and phandles are fixed up using `__fixups__` and `__local_fixups__`.
4. It patches runtime information into the tree:
    - `/chosen` gets `bootargs`, `linux,initrd-start/end` and `kaslr-seed`.
    - `/memory` gets the RAM layout.
    - Serial number and `androidboot.*` values are added (Android 12+ moves most of these to **bootconfig** rather than the cmdline or DT).
5. It jumps to the kernel with the physical address of the final DTB in `x0` on arm64, or `r2` on arm32.

3. Kernel: early scan, unflatten, populate

Early flat scan. This happens before memory allocators exist. `setup_arch()` calls `setup_machine_fdt()`, which calls `early_init_dt_scan()`. That walks the flat blob directly:

- `early_init_dt_scan_root()` reads the root `#address-cells` and `#size-cells`.
- `early_init_dt_scan_chosen()` reads the command line, initrd location and rng seed.
- `early_init_dt_scan_memory()` adds RAM to memblock.
- `early_init_fdt_scan_reserved_mem()` handles `/reserved-memory` regions such as carveouts for the modem, display and TrustZone.

Unflattening. `unflatten_device_tree()` converts the blob into a live tree of `struct device_node` objects. Each node holds a linked list of `struct property` entries and parent/child/sibling pointers. `of_alias_scan()` then processes `/aliases`. From this point everything uses the `of_*` API rather than raw offsets.

Device creation. `of_platform_default_populate_init()`, an `arch_initcall_sync`, calls `of_platform_populate()`. It creates a `platform_device` for every root child that has a `compatible` property. It also recurses into `simple-bus` style nodes. Bus controllers then register their own children: `of_i2c_register_devices()`, SPI, MDIO and so on.

Driver matching and probe. A driver declares an `of_match_table` of compatible strings. When a node's `compatible` matches, `probe()` runs and reads its configuration from the node:

```
static const struct of_device_id my_ids[] = {
    { .compatible = "vendor,my-sensor" }, { }
};

static int my_probe(struct platform_device *pdev)
{
    struct device_node *np = pdev->dev.of_node;
    u32 rate;
    of_property_read_u32(np, "clock-frequency", &rate);
    /* also: devm_clk_get, devm_regulator_get, of_get_named_gpio,
```

```
        platform_get_irq, devm_ioremap_resource ... */
}
```

Resources such as `reg`, `interrupts`, clocks, regulators, GPIOs and pinctrl are resolved through phandles at probe time. If a dependency isn't ready yet, the driver returns `-EPROBE_DEFER`. `fw_devlink` uses the DT phandle links to order probing.

4. Android userspace

The kernel exposes the live tree at `/sys/firmware/devicetree/base`, also reachable as `/proc/device-tree`. The raw blob is at `/sys/firmware/fdt`.

Android `init` reads the node `/proc/device-tree/firmware/android/` (`compatible = "android,firmware"`):

- `ProcessKernelDt()` turns its properties into `ro.boot.*` system properties.
- On Android 8–10 this node also held the **early-mount fstab** and `vbmeta` info used by first-stage init. Newer releases mostly moved fstab into the first-stage ramdisk and boot parameters into bootconfig.

Useful debugging commands

```
# Dump the final merged tree from a running device
adb shell cat /sys/firmware/fdt > live.dtb
dtc -I dtb -O dts live.dtb -o live.dts

# Or decompile from the filesystem view
dtc -I fs -O dts /sys/firmware/devicetree/base

# Inspect a dtbo image
mkdtboimg.py dump dtbo.img -b dtbo_entry
```

To verify that an overlay applied correctly, use AOSP's `ufdt_verify_overlay` tool or the VTS DTBO tests.

Explain in detail for GPU loading from device tree and how probe is done for GPU

The GPU follows the same flow as any other DT-described device. The kernel turns its node into a `platform_device`, and a GPU driver claims it by matching `compatible`. The GPU is unusual in how much it pulls from the DT: several register regions, multiple interrupts, a dozen clocks, power domains, interconnect bandwidth, IOMMU context banks, a frequency/voltage table, speed bins from fuses, and often a separate management microcontroller. I'll use Qualcomm Adreno (the downstream KGSL driver used on most Snapdragon Android phones) as the main example, then show how ARM Mali differs. Exact property names shift between kernel versions and SoCs, so treat the snippets as representative rather than copied from one specific tree.

1. On Android, the GPU driver is a module

With GKI (Generic Kernel Image), the GPU driver isn't built into the kernel. It ships as a vendor module (`msm_kgsl.ko` for Adreno, `mali_kbase.ko` for Mali) in `vendor_boot` or `vendor_dlkm`. The order of events is:

1. At boot, `of_platform_populate()` creates a `platform_device` for the GPU node (and for its GMU and SMMU child nodes). No driver is attached yet.
2. First-stage `init` loads modules in the order listed in `modules.load`. This includes the clock controller (`gpucc`), interconnect, regulator/RPMh and IOMMU modules.
3. When `msm_kgsl.ko` is inserted, its `platform_driver_register()` makes the driver core walk the existing platform devices. The `compatible` string matches and `probe()` runs.
4. If a dependency such as the GPU clock controller isn't ready, probe returns `-EPROBE_DEFER`. The core retries whenever another driver binds. `fw_devlink` reads phandle properties like `clocks`, `interconnects`, `iommus` and `*-supply` to work out these dependencies ahead of time and delays probe until the suppliers exist.

2. Anatomy of a GPU devicetree node (Adreno)

```
gpu: qcom,kgsl-3d0@3d00000 {
    compatible = "qcom,adreno-gpu-a660", "qcom,kgsl-3d0";
    status = "ok";

    /* MMIO regions */
    reg = <0x3d00000 0x40000>, <0x3d61000 0x800>, <0x03de0000 0x10000>;
    reg-names = "kgsl_3d0_reg_memory", "cx_dbgc", "cx_misc";

    /* Interrupt */
    interrupts = <GIC_SPI 300 IRQ_TYPE_LEVEL_HIGH>;
    interrupt-names = "kgsl_3d0_irq";

    /* GPU identity (optional on newer chips, read from HW) */
    qcom,chipid = <0x06060000>;

    /* Clocks provided by the gpucc / gcc clock controllers */
    clocks = <&gcc GCC_GPU_GPLL0_CLK_SRC>, <&gcc GCC_DDRSS_GPU_AXI_CLK>,
             <&gpucc GPU_CC_AHB_CLK>, <&gcc GCC_GPU_MEMNOC_GFX_CLK>,
             <&gpucc GPU_CC_CX_GMU_CLK>;
    clock-names = "gcc_gpu_gpll0", "mem_iface_clk", "gpu_cc_ahb",
                  "mem_clk", "gmu_clk";

    /* Power domains (GDSCs) */
    vddcx-supply = <&gpu_cx_gdsc>;
    vdd-supply   = <&gpu_gx_gdsc>;

    /* GPU → DDR bandwidth path */
    interconnects = <&gem_noc MASTER_GFX3D &mc_virt SLAVE_EBI1>;
    interconnect-names = "gpu_icc_path";

    /* Speed bin fuse, read via nvmem */
    nvmem-cells = <&gpu_speed_bin>;
    nvmem-cell-names = "speed_bin";

    /* Reserved memory for the secure zap shader */
    zap-shader {
```

```
        memory-region = <&gpu_microcode_mem>;
    };

    /* Frequency/voltage/bandwidth tables, one per speed bin */
    qcom,gpu-pwrlevel-bins {
        #address-cells = <1>;
        #size-cells = <0>;

        qcom,gpu-pwrlevels-0 {
            qcom,speed-bin = <0>;
            qcom,initial-pwrlevel = <5>;

            qcom,gpu-pwrlevel@0 {
                reg = <0>;
                qcom,gpu-freq = <840000000>;
                qcom,level = <RPMH_REGULATOR_LEVEL_TURBO_L1>;
                qcom,bus-freq = <11>;
                qcom,bus-min = <10>;
                qcom,bus-max = <11>;
            };
            /* ... lower levels ... */
        };
    };
};

/* GPU Management Unit: a Cortex-M core that handles GPU power */
gmu: qcom,gmu@3d6a000 {
    compatible = "qcom,gpu-gmu";
    reg = <0x3d6a000 0x34000>, <0x3de0000 0x10000>, <0xb290000 0x10000>;
    reg-names = "gmu", "rscc", "gmu_pdc";
    interrupts = <GIC_SPI 304 IRQ_TYPE_LEVEL_HIGH>,
                 <GIC_SPI 305 IRQ_TYPE_LEVEL_HIGH>;
    interrupt-names = "hfi", "gmu";
    iommus = <&kgsl_smmu 0x5 0x400>;
    mboxes = <&qmp_aop 0>;
};

/* IOMMU context banks for GPU address spaces */
kgsl_msm_iommu: qcom,kgsl-iommu@3da0000 {
    compatible = "qcom,kgsl-smmu-v2";
    reg = <0x3da0000 0x40000>;

    gfx3d_user: gfx3d_user {
        compatible = "qcom,smmu-kgsl-cb";
        iommus = <&kgsl_smmu 0x0 0x401>;
    };
    gfx3d_secure: gfx3d_secure {
        compatible = "qcom,smmu-kgsl-cb";
        iommus = <&kgsl_smmu 0x2 0x400>;
    };
};
```

Many of these values are tuned per board through DTBO overlays: the power-level tables, `qcom,initial-pwrlevel`, thermal limits and bus votes. The bootloader merges them before the kernel ever sees the tree.

3. KGSL probe, step by step

Newer KGSL uses the **component framework**. The GPU won't bind until its GMU and SMMU parts have probed, so the GPU device acts as the "master" and the GMU and SMMU are its "components".

3a. Module init and registration

```
static const struct of_device_id adreno_match_table[] = {
    { .compatible = "qcom,kgsl-3d0", .data = &device_3d0 },
    { }
};

static struct platform_driver adreno_platform_driver = {
    .probe  = adreno_probe,
    .remove = adreno_remove,
    .driver = {
        .name = "kgsl-3d",
        .of_match_table = adreno_match_table,
        .pm = &adreno_pm_ops,
    },
};

static int __init kgsl_3d_init(void)
{
    kgsl_core_init();                   /* class, char-dev region, workqueues */
    platform_driver_register(&kgsl_smmu_driver);
    platform_driver_register(&gmu_driver);
    return platform_driver_register(&adreno_platform_driver);
}
```

3b. `adreno_probe()` waits for its components

```
static int adreno_probe(struct platform_device *pdev)
{
    struct component_match *match = NULL;

    /* Add GMU and SMMU nodes as required components */
    adreno_add_components(&pdev->dev, &match);

    /* adreno_bind() runs only once all components are ready */
    return component_master_add_with_match(&pdev->dev,
                                           &adreno_ops, match);
}
```

3c. `adreno_bind()` identifies the GPU

```
static int adreno_bind(struct device *dev)
{
    struct platform_device *pdev = to_platform_device(dev);
    const struct adreno_gpu_core *gpucore;

    /* Read qcom,chipid from DT (or the chip-id register) and look it up
       in the driver's adreno_gpulist[] table */
    gpucore = adreno_identify_gpu(pdev, &chipid);

    /* The table entry supplies the GPU-specific ops (a6xx/a7xx...),
       firmware names (e.g. "a660_sqe.fw", "a660_gmu.bin"),
       GMEM size, register lists, etc. */
    return gpucore->gpudev->probe(pdev, chipid, gpucore);
```

```
}
```

The GPU's identity is split between the DT and the driver. The DT says *which* GPU it is through `chipid` or `compatible`. The driver's built-in `adreno_gpulist[]` then supplies the matching firmware names, register lists, feature flags and generation-specific ops.

3d. Generation-specific probe, then common device probe

`a6xx_gmu_device_probe()` (or the a7xx equivalent) allocates the device structure and calls the shared `adreno_device_probe()`, which then calls `kgsl_device_platform_probe()`. That function parses the rest of the node. The table below shows each step, the DT input it consumes and the kernel API it uses:

Step	Reads from DT	Kernel API used
Map registers	`reg`, `reg-names`	`platform_get_resource_byname()`, `devm_ioremap()`
Interrupt	`interrupts`, `interrupt-names`	`platform_get_irq_byname()`, `devm_request_irq()`
Clocks	`clocks`, `clock-names`	`devm_clk_bulk_get_all()`
Power rails	`vdd-supply`, `vddcx-supply`	`devm_regulator_get()`
Bandwidth	`interconnects`	`of_icc_get()`
Speed bin	`nvmem-cells`	`nvmem_cell_read()`
Power levels	`qcom,gpu-pwrlevel-bins`	`of_property_read_u32()` per child node
DCVS	power levels from above	`devfreq_add_device()`, OPP table built at runtime
IOMMU	`qcom,smmu-kgsl-cb` children	`iommu_domain_alloc()`, attach context banks
Secure memory	`zap-shader/memory-region`	`of_reserved_mem_lookup()`
GMU	GMU node regs, IRQs, mailbox	GMU component bind, HFI queue setup

The speed bin is one detail worth spelling out. The driver reads the fuse value, then walks the `qcom,gpu-pwrlevels-N` children and picks the one whose `qcom,speed-bin` matches. That is how a single DT supports both a binned-down part capped at, say, 680 MHz and a full-speed 840 MHz part of the same chip.

3e. Registering the user interface

Probe finishes with:

- creating the character device **`/dev/kgsl-3d0`**,
- adding the sysfs nodes `/sys/class/kgsl/kgsl-3d0/` (`gpu_model`, `gpuclk`, `max_pwrlevel`, `gpubusy`, ...),
- creating the devfreq node under `/sys/class/kgsl/kgsl-3d0/devfreq/`,
- setting up debugfs and snapshot (GPU crash dump) buffers,
- enabling runtime PM.

The GPU stays **powered off**. Probe only gathers resources.

4. First open: where the GPU actually starts

The heavy hardware work happens later, when userspace first opens `/dev/kgsl-3d0`. On Android this is usually SurfaceFlinger or a graphics HAL loading the vendor GLES/Vulkan driver (`libGLESv2_adreno.so`, `vulkan.adreno.so`). The first open runs this sequence:

1. **GMU boot.** The driver enables the CX GDSC and GMU clocks, then loads `aXXX_gmu.bin` with `request_firmware()` from `/vendor/firmware`. It starts the GMU's Cortex-M core and opens the HFI message queues. From then on the GMU controls the GX power rail and votes RPMh levels on the CPU's behalf.
2. **Zap shader.** Adreno comes out of reset in a secure state. The driver loads the signed zap shader into the reserved `memory-region` and asks TrustZone, through `qcom_scm` / PAS, to run it. That moves the GPU into non-secure mode so the kernel can program it.
3. **SQE/CP microcode.** `aXXX_sqe.fw` is loaded into the command processor.
4. **Hardware init.** The driver programs the hardware-configuration registers from `adreno_gpulist`. It sets up ringbuffers, enables the IRQ, and votes the initial power level and bus bandwidth taken from the DT tables.
5. **Normal operation.** Command submissions arrive through ioctls. When the GPU goes idle, a timer powers it down through runtime PM and GMU-controlled power collapse.

5. Mali (Exynos, MediaTek, Tensor, etc.)

Mali uses the same pattern with a simpler binding:

```
gpu: gpu@13000000 {
    compatible = "arm,mali-valhall";          /* or mali-bifrost / mali-midgard
*/
    reg = <0x13000000 0x4000>;
    interrupts = <GIC_SPI 382 IRQ_TYPE_LEVEL_HIGH>,
                 <GIC_SPI 381 IRQ_TYPE_LEVEL_HIGH>,
                 <GIC_SPI 380 IRQ_TYPE_LEVEL_HIGH>;
    interrupt-names = "JOB", "MMU", "GPU";
    clocks = <&mfgsys CLK_MFG_BG3D>;
    mali-supply = <&vgpu>;
    power-domains = <&spm MT8183_POWER_DOMAIN_MFG>;
    operating-points-v2 = <&gpu_opp_table>;
};

gpu_opp_table: opp-table {
    compatible = "operating-points-v2";
    opp-800000000 { opp-hz = /bits/ 64 <800000000>; opp-microvolt = <750000>; };
    /* ... */
};
```

The `kbase_platform_device_probe()` path runs a table of init steps through `kbase_device_init()`:

1. **Interrupts.** `assign_irqs()` gets the JOB, MMU and GPU IRQs with `platform_get_irq_byname()`.
2. **Registers.** `registers_map()` calls `ioremap` on `reg`.

3. **Power control.** `power_control_init()` gets the regulators and clocks, and attaches the power domains.
4. **GPU identity.** The driver reads the **`GPU_ID` register** from the hardware. Unlike Adreno, Mali doesn't rely on the DT for its identity, and the hardware ID selects the feature and errata tables.
5. **Backend and MMU.** The job manager (or CSF firmware on newer GPUs), the GPU's own MMU, and the power-management state machine are initialized.
6. **DVFS.** `kbase_devfreq_init()` builds DVFS from the standard **OPP table** (`operating-points-v2`), not a vendor-specific format.
7. **User interface.** `kbase_device_misc_register()` creates **`/dev/mali0`**.

SoC vendors hook in through `kbase_platform_funcs` and `power_management_callbacks`. Samsung, MediaTek and Google put their own power sequencing, voltage tables and thermal hooks there, often reading extra vendor-specific DT properties.

For comparison, the upstream drivers (`drm/msm` for Adreno, `panfrost`/`panthor` for Mali) parse nearly the same properties. They expose the GPU as a DRM render node (`/dev/dri/renderD128`) instead of a custom char device, and they use standard OPP tables throughout.

6. Debugging GPU probe on a device

```
# Did the platform device get created from DT?
adb shell ls /sys/bus/platform/devices/ | grep -i -e kgsl -e gpu -e mali

# Did a driver bind to it?
adb shell ls -l /sys/bus/platform/drivers/kgsl-3d/

# Stuck waiting on a dependency?
adb shell cat /sys/kernel/debug/devices_deferred

# Probe logs
adb shell dmesg | grep -i -e kgsl -e adreno -e gmu -e mali

# Check what the DT really contains after overlays
adb shell ls /proc/device-tree/soc/qcom,kgsl-3d0@3d00000/
adb shell cat /sys/class/kgsl/kgsl-3d0/gpu_model
adb shell cat /sys/class/kgsl/kgsl-3d0/devfreq/available_frequencies
```

Common failures and their usual causes:

- **The device never probes.** The node has the wrong `status`, the `compatible` doesn't match, or the module was never loaded because it is missing from `modules.load`.
- **The device is stuck in `devices_deferred`.** The GPU clock controller, interconnect or SMMU module hasn't loaded yet.
- **Probe succeeds but the first open fails.** Usually a missing firmware file in `/vendor/firmware`, or a zap shader or TrustZone failure. Look for "zap" or "PIL"/"PAS" errors in dmesg.
- **The wrong maximum frequency.** The speed-bin fuse didn't match any `qcom,gpu-pwrlevels-N` entry, or a DTBO overlay replaced the table.

```
gpu: qcom,kgsl-3d0@3d00000 {
compatible = "qcom,adreno-gpu-a660", "qcom,kgsl-3d0";
status = "ok";

/* MMIO regions */
reg = <0x3d00000 0x40000>, <0x3d61000 0x800>, <0x03de0000 0x10000>;
reg-names = "kgsl_3d0_reg_memory", "cx_dbgc", "cx_misc";

/* Interrupt */
interrupts = <GIC_SPI 300 IRQ_TYPE_LEVEL_HIGH>;
interrupt-names = "kgsl_3d0_irq";

/* GPU identity (optional on newer chips, read from HW) */
qcom,chipid = <0x06060000>;

/* Clocks provided by the gpucc / gcc clock controllers */
clocks = <&gcc GCC_GPU_GPLL0_CLK_SRC>, <&gcc GCC_DDRSS_GPU_AXI_CLK>,
         <&gpucc GPU_CC_AHB_CLK>, <&gcc GCC_GPU_MEMNOC_GFX_CLK>,
         <&gpucc GPU_CC_CX_GMU_CLK>;
clock-names = "gcc_gpu_gpll0", "mem_iface_clk", "gpu_cc_ahb",
              "mem_clk", "gmu_clk";

/* Power domains (GDSCs) */
vddcx-supply = <&gpu_cx_gdsc>;
vdd-supply   = <&gpu_gx_gdsc>;

/* GPU → DDR bandwidth path */
interconnects = <&gem_noc MASTER_GFX3D &mc_virt SLAVE_EBI1>;
interconnect-names = "gpu_icc_path";

/* Speed bin fuse, read via nvmem */
nvmem-cells = <&gpu_speed_bin>;
nvmem-cell-names = "speed_bin";

/* Reserved memory for the secure zap shader */
zap-shader {
    memory-region = <&gpu_microcode_mem>;
};

/* Frequency/voltage/bandwidth tables, one per speed bin */
qcom,gpu-pwrlevel-bins {
    #address-cells = <1>;
    #size-cells = <0>;

    qcom,gpu-pwrlevels-0 {
        qcom,speed-bin = <0>;
        qcom,initial-pwrlevel = <5>;

        qcom,gpu-pwrlevel@0 {
            reg = <0>;
            qcom,gpu-freq = <840000000>;
            qcom,level = <RPMH_REGULATOR_LEVEL_TURBO_L1>;
            qcom,bus-freq = <11>;
            qcom,bus-min = <10>;
            qcom,bus-max = <11>;
        };
        /* ... lower levels ... */
    };
};

};
```

/* GPU Management Unit: a Cortex-M core that handles GPU power */
gmu: qcom,gmu@3d6a000 {
compatible = "qcom,gpu-gmu";
reg = <0x3d6a000 0x34000>, <0x3de0000 0x10000>, <0xb290000 0x10000>;
reg-names = "gmu", "rscc", "gmu_pdc";
interrupts = <GIC_SPI 304 IRQ_TYPE_LEVEL_HIGH>,
<GIC_SPI 305 IRQ_TYPE_LEVEL_HIGH>;
interrupt-names = "hfi", "gmu";
iommus = <&kgsl_smmu 0x5 0x400>;
mboxes = <&qmp_aop 0>;
};

/* IOMMU context banks for GPU address spaces */
kgsl_msm_iommu: qcom,kgsl-iommu@3da0000 {
compatible = "qcom,kgsl-smmu-v2";
reg = <0x3da0000 0x40000>;

```
gfx3d_user: gfx3d_user {
    compatible = "qcom,smmu-kgsl-cb";
    iommus = <&kgsl_smmu 0x0 0x401>;
};
gfx3d_secure: gfx3d_secure {
    compatible = "qcom,smmu-kgsl-cb";
    iommus = <&kgsl_smmu 0x2 0x400>;
};
```

};

Explain this line by line

This is a **Device Tree (DT) description** of the Adreno 660 GPU subsystem on a Qualcomm Snapdragon SoC (the A660 shipped in the Snapdragon 888 / SM8350, "Lahaina"). It is written for Qualcomm's downstream **KGSL** driver (Kernel Graphics Support Layer), not the upstream `msm`/freedreno driver, which uses different bindings. The kernel reads this at boot to learn where the GPU's registers live, which clocks, power rails and memory paths it needs, and what frequencies it can run at.

There are three nodes: the GPU itself, the GMU that manages its power, and the IOMMU that gives it virtual memory.

---

1. The GPU node

```
gpu: qcom,kgsl-3d0@3d00000 {
```

`gpu:` is a **label**, so other nodes can reference this one as `&gpu`. `qcom,kgsl-3d0` is the node name, and `@3d00000` is the unit address, which by convention matches the first `reg` entry. "3d0" means the first 3D graphics core.

```
    compatible = "qcom,adreno-gpu-a660", "qcom,kgsl-3d0";
```

This is the string the kernel matches against drivers, listed from most specific to most generic. The KGSL driver binds to `qcom,kgsl-3d0`, and the A660 string lets it pick A660-specific code paths.

```
status = "ok";
```

Enables the device. `"okay"` is the canonical spelling, but `"ok"` is also accepted. A value of `"disabled"` would make the kernel skip it.

```
reg = <0x3d00000 0x40000>, <0x3d61000 0x800>, <0x03de0000 0x10000>;
reg-names = "kgsl_3d0_reg_memory", "cx_dbgc", "cx_misc";
```

These are three MMIO regions, each given as `<physical-base size>`:

- `kgsl_3d0_reg_memory` at 0x3D00000 is 256 KB and holds the main GPU register space: the command processor, the ringbuffer control and the RBBM.
- `cx_dbgc` at 0x3D61000 is 2 KB of debug-controller registers in the always-on CX domain. These are used for GPU snapshots and crash dumps.
- `cx_misc` at 0x3DE0000 is 64 KB of miscellaneous CX-domain registers.

`reg-names` lets the driver look regions up by name instead of index.

```
interrupts = <GIC_SPI 300 IRQ_TYPE_LEVEL_HIGH>;
interrupt-names = "kgsl_3d0_irq";
```

This is the GPU's main interrupt line to the ARM GIC: Shared Peripheral Interrupt 300, which is GIC hardware IRQ 332 since SPIs start at 32. It is level-triggered and active-high. It fires for things like "command batch done", hangs and faults.

```
qcom,chipid = <0x06060000>;
```

The GPU identity, encoded one byte per field as core.major.minor.patch, so this reads 6.6.0.0, meaning A660. The comment says newer chips read this from hardware. Here it acts as an override or fallback. If your silicon reports a different patch level (for example 0x06060001), a mismatch here can select the wrong firmware or quirks.

```
clocks = <&gcc GCC_GPU_GPLL0_CLK_SRC>, <&gcc GCC_DDRSS_GPU_AXI_CLK>,
         <&gpucc GPU_CC_AHB_CLK>, <&gcc GCC_GPU_MEMNOC_GFX_CLK>,
         <&gpucc GPU_CC_CX_GMU_CLK>;
clock-names = "gcc_gpu_gpll0", "mem_iface_clk", "gpu_cc_ahb",
              "mem_clk", "gmu_clk";
```

Each entry is `<&provider clock-id>`, where `gcc` is the Global Clock Controller and `gpucc` is the GPU Clock Controller:

- `gcc_gpu_gpll0` is the GPLL0 PLL output routed to the GPU clock controller, used as a source.
- `mem_iface_clk` is the AXI clock between the DDR subsystem and the GPU.
- `gpu_cc_ahb` is the AHB bus clock, needed for the CPU to access GPU registers at all.
- `mem_clk` is the GPU's clock into the memory NoC.
- `gmu_clk` clocks the GMU microcontroller.

The GPU core clock (GX) is deliberately absent. On A6xx-class GPUs the GMU owns GX clock and voltage scaling, so Linux never touches it directly.

```
vddcx-supply = <&gpu_cx_gdsc>;
vdd-supply   = <&gpu_gx_gdsc>;
```

These are the two power domains, which downstream kernels expose as regulators. A GDSC is a Globally Distributed Switch Controller, a hardware power switch.

- CX GDSC powers the "always-ish on" side: the GMU, debug logic and register interface.
- GX GDSC powers the actual shader and rendering core, and is switched off whenever the GPU is idle.

```
interconnects = <&gem_noc MASTER_GFX3D &mc_virt SLAVE_EBI1>;
interconnect-names = "gpu_icc_path";
```

This declares the GPU's path to DRAM: from the GPU as a master on the GEM NoC (the graphics/compute memory network-on-chip) to EBI1, which is external DDR, via the memory controller. The driver votes bandwidth on this path so DDR runs fast enough for the current workload.

```
nvmem-cells = <&gpu_speed_bin>;
nvmem-cell-names = "speed_bin";
```

This reads a **fuse** (via the nvmem framework, usually backed by QFPROM) that records how the chip was binned at the factory. Better-binned chips can run faster or at lower voltage. The value selects which power-level table below is used.

```
zap-shader {
    memory-region = <&gpu_microcode_mem>;
};
```

The zap shader is signed firmware loaded through TrustZone into a reserved memory carveout (`gpu_microcode_mem`, defined under `/reserved-memory`). It lets the GPU safely switch between secure (DRM-protected content) and non-secure modes. Without it, the GPU cannot leave secure mode after boot and will usually fail to initialise.

Power-level table

```
qcom,gpu-pwrlevel-bins {
    #address-cells = <1>;
    #size-cells = <0>;
```

This is a container for one table per speed bin. `#address-cells = 1` and `#size-cells = 0` mean child `reg` values are plain single-number indices, not address ranges.

```
        qcom,gpu-pwrlevels-0 {
            qcom,speed-bin = <0>;
            qcom,initial-pwrlevel = <5>;
```

This table applies when the fused speed bin equals 0. The GPU starts at power level 5, a moderate level (lower numbers are faster), and DCVS (dynamic clock and voltage scaling) moves it from there.

```
            qcom,gpu-pwrlevel@0 {
                reg = <0>;
                qcom,gpu-freq = <840000000>;
                qcom,level = <RPMH_REGULATOR_LEVEL_TURBO_L1>;
                qcom,bus-freq = <11>;
```

```
            qcom,bus-min = <10>;
            qcom,bus-max = <11>;
        };
```

Level 0 is the fastest operating point:

- `gpu-freq` is the core clock, here 840 MHz.
- `level` is the voltage corner voted for the GX rail through RPMh, Qualcomm's hardware resource manager. TURBO_L1 is one of the highest corners.
- `bus-freq` is the default DDR bandwidth level used at this GPU frequency. It is an index into the DDR bandwidth table, not a value in Hz.
- `bus-min` and `bus-max` set the range the bus vote may float within while at this GPU level.

Lower levels (1, 2, …) follow with progressively lower frequency, voltage and bandwidth.

---

2. The GMU node

```
gmu: qcom,gmu@3d6a000 {
    compatible = "qcom,gpu-gmu";
```

The **GPU Management Unit** is a small Cortex-M class core next to the GPU. It runs Qualcomm firmware that powers GX on and off, sets GPU frequency and voltage, and handles fast idle transitions far quicker than Linux could.

```
    reg = <0x3d6a000 0x34000>, <0x3de0000 0x10000>, <0xb290000 0x10000>;
    reg-names = "gmu", "rscc", "gmu_pdc";
```

These are three register regions:

- `gmu` holds the GMU's own registers and its interface memory.
- `rscc` is the Resource State Coordinator. It saves and restores state and sends RPMh votes when the GPU sleeps.
- `gmu_pdc` is the Power Domain Controller. It sequences the hardware sleep and wake of the GPU's power island.

```
    interrupts = <GIC_SPI 304 IRQ_TYPE_LEVEL_HIGH>,
                 <GIC_SPI 305 IRQ_TYPE_LEVEL_HIGH>;
    interrupt-names = "hfi", "gmu";
```

`hfi` is for the Host Firmware Interface, the message queue between the Linux driver and GMU firmware ("set level 3", "ack"). `gmu` is for GMU-side events and errors, such as watchdog or firmware faults.

```
    iommus = <&kgsl_smmu 0x5 0x400>;
```

The GMU accesses memory through the GPU's SMMU using **Stream ID 0x5** with mask 0x400. Mask bits are "don't care", so SIDs 0x5 and 0x405 both match. This gives it an isolated address space for its firmware and HFI queues.

```
    mboxes = <&qmp_aop 0>;
```

This is a mailbox channel to the **AOP** (Always-On Processor), using the QMP protocol. KGSL uses it to send configuration messages to AOP, for example enabling ACD (adaptive clock distribution) or GPU-related power settings.

---

3. The IOMMU node

```
kgsl_msm_iommu: qcom,kgsl-iommu@3da0000 {
    compatible = "qcom,kgsl-smmu-v2";
    reg = <0x3da0000 0x40000>;
```

This is KGSL's description of the GPU's dedicated SMMU (ARM SMMUv2), at 0x3DA0000 with 256 KB of registers. The GPU programs its own page-table switches through this, which is how each process gets a separate GPU address space without a CPU round-trip.

```
    gfx3d_user: gfx3d_user {
        compatible = "qcom,smmu-kgsl-cb";
        iommus = <&kgsl_smmu 0x0 0x401>;
    };
```

This is the **context bank for normal (non-secure) GPU work**, with Stream ID 0x0 and mask 0x401. It therefore matches SIDs 0x0, 0x1, 0x400 and 0x401, which covers the GPU's various non-secure traffic streams. All regular app rendering is translated through this context.

```
    gfx3d_secure: gfx3d_secure {
        compatible = "qcom,smmu-kgsl-cb";
        iommus = <&kgsl_smmu 0x2 0x400>;
    };
```

This is the **secure context bank** with Stream ID 0x2, used when rendering protected content (for example DRM video). Its page tables are controlled by TrustZone, so normal-world software can't read those buffers.

Note that `&kgsl_smmu` refers to the underlying ARM SMMU hardware node defined elsewhere. The `kgsl_msm_iommu` node is KGSL's own wrapper around it.

Things worth double-checking

Overlapping registers. `cx_misc` on the GPU and `rscc` on the GMU both claim **0x3DE0000 + 0x10000**. Two devices mapping the same range is usually a copy-paste error, and the kernel may refuse the second `request_mem_region()`. In SM8350 downstream trees these normally sit at separate addresses, so verify both against your SoC's memory map.

Chip ID. `qcom,chipid` should match the actual silicon revision, or the wrong firmware or quirks may load.

Bus indices. The `qcom,bus-*` values are indices into a DDR bandwidth table. That table must exist (defined elsewhere in the DT or derived by the driver) and have at least 12 entries for index 11 to be valid.

If you tell me what you're doing with this (bringing up a board, porting to mainline freedreno, debugging a GPU that won't probe), I can point you at the specific properties most likely to matter.

Give a usecase of MMIO in Andriod and explain it

MMIO in Android: controlling a hardware peripheral from an app

Memory-Mapped I/O (MMIO) is a technique where a hardware device's control registers are assigned addresses in the CPU's physical address space. The CPU talks to the device with ordinary load and store instructions on those addresses, instead of using special I/O instructions. On ARM-based Android phones, almost every peripheral on the SoC (System-on-Chip) works this way: GPIO, display controller, GPU, camera ISP, UART, I2C/SPI controllers, timers, and so on. ARM has no separate "port I/O" space, so MMIO is essentially the only way to drive hardware.

Use case: turning on the flashlight or vibrator

Say a user taps the flashlight toggle, or an app calls `Vibrator.vibrate()`. Eventually a physical pin on the SoC must change state, for example to switch on an LED driver or a haptic motor. That pin is controlled by a **GPIO controller**, whose registers sit at a fixed physical address, such as `0x0340_0000`, as defined in the SoC's datasheet and the device tree.

The request passes through Android's layers like this:

```
App  →  Framework (CameraManager / VibratorManager)
     →  Binder IPC to system service
     →  HAL (AIDL/HIDL implementation, vendor code)
     →  sysfs / ioctl on a device node (/sys/class/leds/..., /dev/...)
     →  Linux kernel driver
     →  MMIO register write  →  GPIO pin goes HIGH  →  LED/motor turns on
```

Only the last step, inside the kernel driver, is MMIO. A simplified driver looks like this:

```
#include <linux/io.h>
#include <linux/platform_device.h>

#define GPIO_DIR_OFFSET   0x04   // direction register
#define GPIO_DATA_OFFSET  0x08   // output data register
#define FLASH_PIN         (1 << 12)

static void __iomem *gpio_base;
```

```
static int flash_probe(struct platform_device *pdev)
{
    struct resource *res = platform_get_resource(pdev, IORESOURCE_MEM, 0);

    /* Map the physical register block (from device tree) into kernel virtual
memory */
    gpio_base = devm_ioremap_resource(&pdev->dev, res);
    if (IS_ERR(gpio_base))
        return PTR_ERR(gpio_base);

    /* Configure pin 12 as output: read-modify-write the direction register */
    u32 dir = readl(gpio_base + GPIO_DIR_OFFSET);
    writel(dir | FLASH_PIN, gpio_base + GPIO_DIR_OFFSET);
    return 0;
}

static void flash_set(bool on)
{
    u32 val = readl(gpio_base + GPIO_DATA_OFFSET);
    if (on)
        val |= FLASH_PIN;
    else
        val &= ~FLASH_PIN;
    writel(val, gpio_base + GPIO_DATA_OFFSET);   // this store reaches the
hardware
}
```

What is happening

1. **The address comes from the device tree.** The board's `.dts` file declares something like `reg = <0x03400000 0x1000>;` for the GPIO block. The kernel reads this at boot and gives it to the driver as a memory resource.
2. **`ioremap` creates a mapping.** The CPU runs with the MMU on, so the driver can't use the physical address directly. `ioremap` creates a kernel virtual mapping to that physical range. It marks the mapping as **device memory (uncached, strongly ordered)**, so writes are not held in the CPU cache or reordered. Without this, a write might sit in cache and never reach the hardware.
3. **`readl`/`writel` access the registers.** These helpers do 32-bit loads and stores and include the memory barriers that ARM needs. The SoC's bus interconnect sends a store to

`gpio_base + 0x08` to the GPIO controller instead of RAM. The controller latches the value, and the pin voltage changes.

4. **Read-modify-write protects other pins.** One data register controls 32 pins. The driver reads the register, changes only bit 12, and writes it back, so other pins (perhaps controlling other hardware) are left alone. Real drivers also hold a spinlock here, because two CPUs doing this at the same moment could overwrite each other's changes.

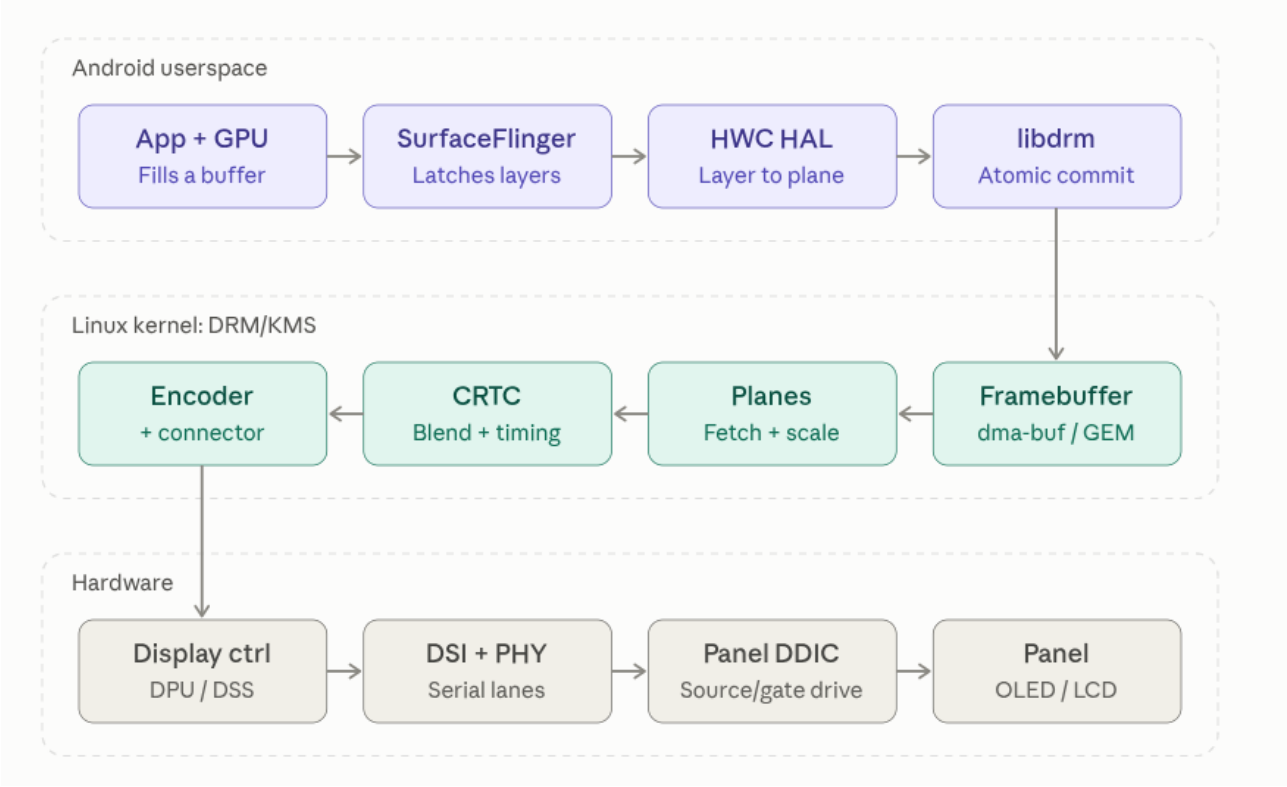
Why the app can't do this directly

In Android, MMIO is deliberately kept inside the kernel. User-space processes have no mapping to device physical addresses, `/dev/mem` is disabled or blocked by SELinux on production builds, and apps are sandboxed. The app asks for "flashlight on" through a framework API, and only the privileged kernel driver touches the register. This protects security, since direct register access could bypass permissions or damage hardware, and it protects portability, since the same app works on Qualcomm, MediaTek, Exynos, or Tensor chips with completely different register maps. Each vendor's HAL and kernel driver hides those differences.

Other places MMIO appears in Android

The same pattern drives much heavier hardware. The **GPU driver** (Adreno, Mali) writes commands and doorbell registers through MMIO to start rendering. The **display controller** gets its frame-buffer addresses and timing configuration through MMIO registers when SurfaceFlinger composes the screen. The **camera ISP** and **video codecs** are set up through MMIO, while bulk data such as image buffers moves through DMA in shared memory. In all these cases, MMIO is the control channel: small register writes that tell the hardware what to do. The large data usually travels separately through DMA.

Explain Display pipeline from Direct Rendering Manager to Display controller to Display in Android



1. Userspace: from app to an atomic commit

An app draws with the GPU (via Skia/HWUI, Vulkan or GLES) into a graphic buffer from its `BufferQueue`. Gralloc allocates that buffer as a dma-buf, often in a compressed, tiled layout such as AFBC on Mali/Exynos or UBWC on Qualcomm. When the GPU finishes, the buffer comes with an *acquire fence* that signals once rendering is complete.

SurfaceFlinger wakes up on vsync and latches the newest buffer from every visible layer, including status bar, app, navigation bar and video. It passes the layer list to the Hardware Composer HAL (HWC2, now the AIDL `IComposer`). In the validate step, HWC decides which layers the display hardware can scan out directly. These get *device composition*. Any layers it cannot handle are flattened by SurfaceFlinger's RenderEngine on the GPU into one *client target* buffer. Typical reasons for falling back to the GPU are too many layers, an unsupported format or a rotation the hardware can't do.

The HWC implementation then talks to the kernel. This is either a vendor HWC or the open-source `drm_hwcomposer`. It imports each dma-buf with `drmPrimeFDToHandle`, wraps it with `drmModeAddFB2WithModifiers`, and builds one atomic request that sets properties on DRM objects. Examples are `FB_ID`, `SRC_*`/`CRTC_*` rectangles, `zpos`, `alpha`, `pixel blend mode`, `IN_FENCE_FD` and the CRTC's `OUT_FENCE_PTR`. HWC can send this request with `DRM_MODE_ATOMIC_TEST_ONLY` to ask "can you do this?" without touching hardware, which is how validation is done. It then sends it for real in `presentDisplay`.

2. Kernel: the DRM/KMS object model

KMS (kernel mode setting) describes any display pipeline with a fixed set of abstract objects. Each SoC vendor's driver maps those objects onto its own silicon:

A framebuffer (`drm_framebuffer`) is just memory: a GEM/dma-buf handle plus format, pitch and modifier. A plane is a hardware fetch-and-process pipe. It reads a framebuffer from DRAM, crops it, scales it, converts colour (YUV → RGB) and places it at a position. Phones expose many planes: primary, several overlays and a cursor. A video layer can therefore go straight to a plane without GPU composition, which saves a lot of power.

The CRTC is the heart of the pipeline. It blends all attached planes in z-order into a single pixel stream and owns the display mode, meaning resolution, refresh rate, porches and sync widths. It also runs the timing generator that produces vblank. Colour management properties such as degamma, CTM and gamma LUTs also live here.

The encoder converts that pixel stream into a specific link protocol (DSI, DP, HDMI), often through chained `drm_bridge` drivers. The connector represents the physical sink and reports its modes. For a phone panel these modes come from a `drm_panel` driver, which also sends the panel's power-on and initialization command sequence. An external monitor reports its modes through EDID instead.

When the atomic commit arrives, the driver runs `atomic_check`, which validates bandwidth, scaler limits and clocks. It then waits on every in-fence and writes the new state into the controller's double-buffered *shadow registers*. Nothing changes on screen yet. The hardware latches the new configuration at the next vblank (or "flush" in command mode), so a frame never tears halfway. At that point the driver sends a vblank/flip-complete event and signals the out-fence.

3. The display controller hardware

The display controller is the block actually doing the work. Examples are Qualcomm DPU/MDSS, Samsung DECON, MediaTek OVL/RDMA, TI DSS and ARM Mali-DP. Its internal stages mirror the KMS objects closely:

The source pipes (Qualcomm calls them SSPP) are DMA engines that fetch each plane's buffer from DRAM, decompress UBWC/AFBC, scale and convert format. The layer mixer alpha-blends the pipes in z-order, which corresponds to the CRTC's blending role. The post-processing block applies panel-wide colour work: gamma, colour correction matrices, night-light, HDR tone mapping, dithering and sometimes local-contrast enhancement. Display Stream Compression (DSC) can then compress the stream so that a 1440p 120 Hz image fits through limited DSI bandwidth. Finally the interface/timing generator turns the result into a timed pixel stream for the DSI controller.

The DSI host packs pixels into MIPI DSI packets and drives them over a D-PHY or C-PHY, usually 4 lanes, across the flex cable to the panel. There are two DSI modes. In video mode the controller streams pixels continuously with timing signals, like a classic monitor. In command mode the panel's driver IC (DDIC) has its own frame memory (GRAM). The SoC sends a new frame only when something changes, and the panel signals the right moment with a tearing-effect (TE) line. Command mode is a big power saver for static content.

4. The panel

The DDIC on the panel receives the DSI stream, decompresses DSC if used, and stores or forwards each line. It drives the source drivers (column voltages, which set pixel brightness) and the gate drivers (which select one row at a time) to refresh the OLED or LCD matrix. It also handles panel-level features: brightness/DBV, variable refresh (LTPO panels dropping to 1–10 Hz), AOD low-power modes and the OLED compensation tables applied in the factory.

5. How it all stays in sync

Two loops tie the layers together. The vsync loop starts with the panel's TE signal or the controller's timing generator, which raises a vblank interrupt. DRM forwards it as a vblank event. HWC reports it through its vsync callback, and SurfaceFlinger's `VsyncSchedule`/`Choreographer` uses it to wake apps for the next frame.

The fence loop keeps buffers safe without the CPU waiting. Acquire fences stop the controller from scanning out a buffer the GPU is still writing. The release fence tells the app when the controller has finished reading its old buffer, so the buffer can be reused. The present fence (the KMS out-fence) marks the exact moment a frame became visible. Frame-timing tools like Perfetto use the present fence to detect janky frames.

In short, an app frame typically takes about two or three vsync periods to reach the glass. It is rendered in one, composed and committed in the next, and latched and scanned out at the vblank after that. Most of Android's display performance work, such as more overlay planes, compressed formats, command mode, VRR and DSC, is about doing that path with less GPU work and less memory bandwidth.